Query | Docs | HERY

`jq` and `JSONPATH` are great utilities to extract information from json content. But with the added dimensions of relational entities it becomes a bit too limiting and hard to only use these tools.

So for that reason HERY comes with its own query language that is even simpler than SQL or `jq` and `JSONPATH`.

Selecting all entities

```
entities
```

Yes, that simple!

Selecting by `_entity` property

```
entities._entity("github.com/AmadlaOrg/EntityApplication@v*")
```

Again, yes, that simple!

> In `("")` it is possible to use glob.

Selecting by any entity property

```
entities._entity("github.com/AmadlaOrg/Entity@v*").name("EntityWebserver")
```

This will return all the entity web servers.

```
entities._entity("github.com/AmadlaOrg/Entity@v*").name("EntityWebserver").server_name("example.com")
```

This one will return the web server with server name *example.com*.

Piping?

In UNIX systems to pass data to the next command you pipe with `|` but since this might cause conflicts in the command line the piping character for HERY is: `!`. Just like gstreamer.

When is it possible to use `jq` and `JSONPATH` ?

After selecting one or many entities it is then possible to use `jq` and `JSONPATH` . Since the output from HERY queries are always in JSON format. It is also possible with a flag to indicate to output everything in YAML.

```
entities._entity("github.com/AmadlaOrg/Entity@v*").name("EntityWebserver") ! jq(".[0]") ! jsonpath("$.
```

The documentation for `jq` and `JSONPATH` are found on their respected websites. For HERY queries there isn't more to it than what was shown.

Examples

Get all the entities

```
entities
```

JSON:

```
[
  {
    "_id": "1d286661-b922-4bf4-a317-5530abfe57d5",
    "_entity": "github.com/AmadlaOrg/Entity@v1.0.0",
    "version": "v1.0.0"
  },
  {
    "_id": "5b9efb38-be80-4eeb-9167-1747b52ca72b",
    "_entity": "github.com/AmadlaOrg/EntityApplication@v1.0.0",
    "version": "v1.0.0"
  }
]
```

Table:

```
+--------------------------------------+-----------------------------------------------+---------+
| _id                                  | _entity                                       | Version |
+--------------------------------------+-----------------------------------------------+---------+
| 1d286661-b922-4bf4-a317-5530abfe57d5 | github.com/AmadlaOrg/Entity@v1.0.0            | v1.0.0  |
| 5b9efb38-be80-4eeb-9167-1747b52ca72b | github.com/AmadlaOrg/EntityApplication@v1.0.0 | v1.0.0  |
+--------------------------------------+-----------------------------------------------+---------+
```

Get entities property

```
entities.property("_id")
```

JSON:

```
[
  {
    "_id": "1d286661-b922-4bf4-a317-5530abfe57d5"
  },
  {
    "_id": "5b9efb38-be80-4eeb-9167-1747b52ca72b"
  }
]
```

Table:

```
+--------------------------------------+
| _id                                  |
+--------------------------------------+
| 1d286661-b922-4bf4-a317-5530abfe57d5 |
| 5b9efb38-be80-4eeb-9167-1747b52ca72b |
+--------------------------------------+
```

Get entities multiple property

```
entities.property("_id", "Version")
```

JSON:

```
[
  {
    "_id": "1d286661-b922-4bf4-a317-5530abfe57d5",
    "version": "v1.0.0"
  },
  {
    "_id": "5b9efb38-be80-4eeb-9167-1747b52ca72b",
    "version": "v1.0.0"
  }
]
```

Table:

```
+--------------------------------------+---------+
| _id                                  | Version |
+--------------------------------------+---------+
| 1d286661-b922-4bf4-a317-5530abfe57d5 | v1.0.0  |
| 5b9efb38-be80-4eeb-9167-1747b52ca72b | v1.0.0  |
+--------------------------------------+---------+
```

Get entity by filter

```
entities.contains("_id", "5530")
```

JSON:

```
[
  {
    "_id": "1d286661-b922-4bf4-a317-5530abfe57d5",
    "_entity": "github.com/AmadlaOrg/Entity@v1.0.0",
    "version": "v1.0.0"
  }
]
```

Table:

```
+--------------------------------------+-----------------------------------------------+---------+
| _id                                  | _entity                                       | Version |
+--------------------------------------+-----------------------------------------------+---------+
| 1d286661-b922-4bf4-a317-5530abfe57d5 | github.com/AmadlaOrg/Entity@v1.0.0            | v1.0.0  |
+--------------------------------------+-----------------------------------------------+---------+
```

Planned (not yet implemented):

```
entities.and(contains("_id", "17"), contains("_entity", "Entity@v1.0.0"))
```

```
entities.or(contains("_id", "17"), contains("_entity", "Entity@v1.0.0"))
```

```
entities.contains("_entity", "Entity@v1.0.0")._body
```

JSON:

```
[
  {
    "server_name": "amadla.com",
    "directory": "/var/www/",
    "listen": [
      {
          "ports": [80, 443]
      }
    ]
  }
]
```

Implemented Functions

Function	Description
contains(<property name>, <filter with>)	Filter rows where a column's value contains the given substring
property(<property name> [, ...])	Select specific properties from entities
jq()	Used to query JSON content with jq

Planned Functions

The following functions are not yet implemented. See roadmap.md for details.

Function	Description
equal(<property name>, <filter with>)	
like(<property name>, <filter with>)	Used for pattern matching in string comparisons, often with wildcards (% for any sequence of characters and _ for a single character)
in(<property name>, <values>...)	Used to filter rows where a column's value matches any value in a specified list
not_equal(<property name>, <filter with>)	
not_like(<property name>, <filter with>)	
not_in(<property name>, <values>...)	
and(<functions>...)	
or(<functions>...)	
limit(<number>)	Limit specifies the maximum number of rows to return in a query
offset(<number>)	Offset specifies the number of rows to skip before starting to return results in a query
order_by(<property name> [, <ASC|DESC>])	Using a property to order the output and optionally choose `DESC` or `ASC`
group_by(<property name> [, <property name&gt...])	Group a property or multiple-properties